Chapter 4

Boosted Decision Trees and XGBoost

The newly-proposed trigger and complementary offline selection – discussed in chapters 5 and 6, respectively – are based on a machine learning construct called a Boosted Decision Tree (BDT). This ML method provides a robust and relatively simple algorithm for binary classification problems, which works well both with the HLT and with subsequent offline classification and event selection. The theoretical details and structure of BDTs is described in section 4.1. The software package XGBoost (short for eXtreme Gradient Boosting) will be used for both the creation and evaluation of the BDTs employed at the trigger and offline analysis levels. This software is described in more detail in section 4.2.

4.1 Tree Boosting Basics

The original description of boosted decision trees and gradient descent boosting is described in "Greedy Function Approximation: A Gradient Boosting Machine" by Jerome H. Friedman [31]. The following section is largely based on this seminal paper, with emphasis placed on the facets of the technique most relevant to this paper.

At its core, a binary classification problem is a function estimation problem; a function must be constructed that maps some input values onto a single output score. The general parlance is to label the input variables for a single event as $\vec{x} = \{x_1, x_2, ...x_n\}$, where n is the total number of selected input values. The output, however, is a single score, usually labeled y. There exists

some theoretical function, $F^*(\vec{x})$ that directly maps $\vec{x}$ onto y. This function cannot be known exactly for problems of moderate complexity, so an estimate of this function, $\hat{F}(\vec{x})$, is the goal of the gradient boosting process.

To quantify how closely the approximation $\hat{F}(\vec{x})$ follows the true $F^*(\vec{x})$, some loss function must be defined. There are many forms of loss function, as will be described in section 4.2.1. The loss function is termed $L(y, F(\vec{x}))$, meaning that it quantifies how closely the approximate points, $F(\vec{x})$, follow the true classification values of the events, y. Given these definitions, the goal of the BDT algorithm can be written as given by Friedman [31].

$$F^* = \min_F E_{y,\vec{x}}[L(y, F(\vec{x}))] \tag{4.1}$$

Functional approximation is a rich and complex field with many possible approaches. So-called "additive" approaches often tend to be both conceptually and computationally simple. In this approach, the total mapping function can be built iteratively as a sum of many simpler functions with well-understood parameters. These functions can be combined as a linear combination with a weight, β, applied to each of the simple functions. The generic functions being added together, in the case of a boosted decision tree, is an individual regression tree [31]. Terming these generic regression functions as $h(\vec{x}, \vec{a_m})$, where $\vec{a_m}$ are the chosen parameters for the individual regression tree allows the function defined in 4.1 to be rewritten as given in equation 4.2.

$$F(\vec{x}; \{\beta_m, \vec{a_m}\}) = \sum_{m=1}^{M} \beta_m h(\vec{x}; \vec{a_m}) \tag{4.2}$$

So, then, the goal of the BDT algorithm is no longer to find a functional form that best approximates the mapping function between the inputs and an output score. Instead, with the functional form of h selected as a regression tree, the parameters for each tree, $\vec{a_m}$, and the weights used in adding the trees together, β_m, must be optimized. This addative approach

underpins the methods used by the software package XGBoost, described in the next section.

4.2 XGBoost Software and Binary Classification

XGBoost is a standard software package for ML methods. Specifically, an HLT must generate a decision on whether to keep or discard and event being evaluated. As a result, triggering is a binary classification problem. The goal, then, is to map the input features for each event, $\vec{x}_i$, onto a label for each event, y_i. For a binary classification problem, these y_i values are a label of 0 or 1, denoting the two possible classes (in the case of triggering, these cases are 0 for background and 1 for signal). To arrive at a final yes or no decision on each event, the BDT must generate a prediction, $\hat{y}_i$, for each event given the input features $\vec{x}_i$. Within XGBoost, this prediction can be given as a score between 0 and 1. This score can be understood as the probability that the chosen event is in the signal class. A value of 0 means a predicted 0% probability of the event being signal (thereby meaning the event is certainly background), whereas a score of 1 means a predicted 100% probability of the event being signal. In practice, most scores will lie somewhere between these two extremes. As a result, some threshold can be placed on these output scores to separate signal and background. So, even though XGBoost will output a continuous score between 0 and 1, this threshold (called a score cut) converts this continuous variable into a binary decision on whether to keep or discard and event.

As with the description in the previous section, XGBoost uses a linear combination of so-called decision trees. The full space of possible decision trees is often referred to as the set of classification and regression trees (CART) [31]. Individual instances of functions within the CART can be denoted $f_k(\vec{x})$, where k ranges the number of functions employed, and the vector $\vec{x}$ still contains all of the input variables chosen for the model. The output decision scores can then be mathematically expressed as given in equation 4.3 (note the similarity to equation 4.2). Of note is the fact that this value is not explicitly normalized; in fact, these predictions may sum to any arbitrary positive or negative value [32]. Normalization and transformation of these scores

into a probability must be done to provide the appropriate output, which is handled by XGBoost and the SciKit-Learn package.

$$\hat{y}_i = \sum_{k=1}^{K} f_k(\vec{x}_i) \tag{4.3}$$

While a very simple task may be accomplished with a single decision tree, a complex model can be constructed with what is known as a tree ensemble. In order to understand this linear combination of decision trees, of course, the individual decision tree must first be understood. A very basic schematic approach can first be taken. A decision tree is so called due to its branching structure. Essentially, a decision tree functions like a flow chart. Each "node" or "leaf" of the tree is essentially a question with a yes or no answer. If the answer is yes, the chart splits in one direction to the next node whereas an answer of no splits the chart to a different node. This branching structure allows relatively complex decisions to be made with relatively simple individual nodes. The more layers of nodes are nested together, the more complex the model becomes. In figure 4.1, a very simple schematic of a two-layer decision tree can be seen. In this schematic, the input features, $\vec{x}_i$, are probed by each node. The features for this tree could be denoted as $\vec{x}_i = \{\text{age, gender}\}$.

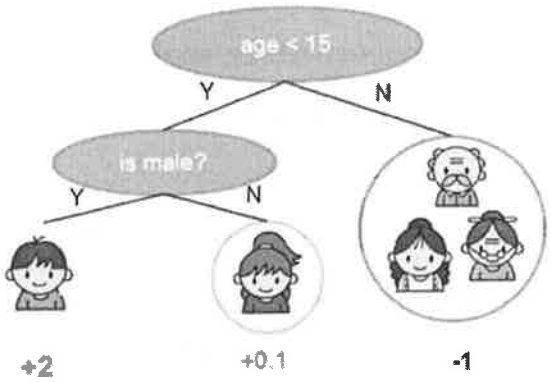


Figure 4.1. A very simple schematic of a single decision tree. Each question, called a node, employs an individual feature of the input data to determine which of the next nodes should be traversed. The weights applied by the nodes can be seen under each classification group. Reproduced from [33].

This schematic shows not only the basic structure of a decision tree, but also the mathematical weights assigned by the decision tree. So, then, by using the inputs $\vec{x}_i$, the score of the tree can then be given by $\hat{y}_{i,k} = f_k(\vec{x}_i)$ as in equation 4.3. The ensemble will consist of a number of such trees, such that the total weight will be assigned by the linear combination of individual tree weights. Schematically, this can be shown in figure 4.2, which has been reproduced from [33].

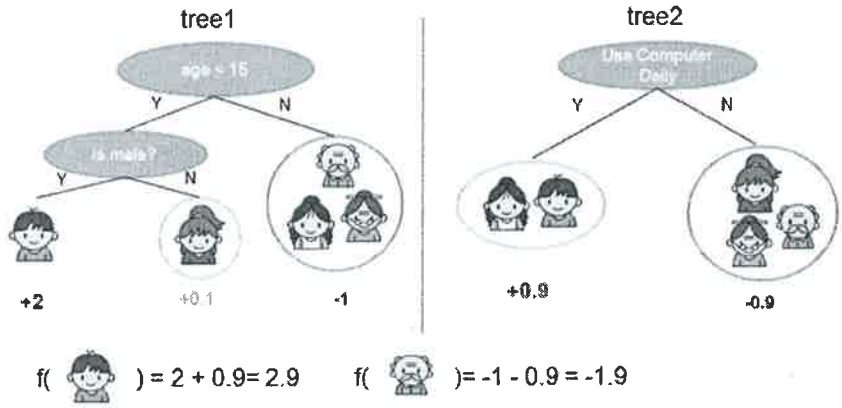


Figure 4.2. A very simple schematic of an ensemble of two decision trees. Each tree assigns a weight given by $f_k\vec{x}_i$, and these weights can be summed for each input member, i, to render the final event score. Reproduced from [33].

Of course, the determination of the weights at each node must be optimized. In addition, the number of trees and the overall comlplexity of these trees must be quantified and optimized for the specific problem at hand. This leads to the implementation of an objective function, described in the next section.

4.2.1 The Objective Function

While the loss function alone, employed in equation 4.1, is a good starting point for function approximation, it does not capture all the necessary complexity of a software-based algorithm. Instead, a so-called objective function can be constructed [33] [32]. The objective function begins from a chosen loss function as before. An additional term, called a regularization term, is added to the objective function to penalize overly-complex models. This works toward

two aims: so-called overfitting must be avoided, as finding a final functional form that too closely fits the input data reduces the generalizability of the model; the penalization of model complexity further works to ensure that the software selects the simplest possible tree structure to achieve the desired accuracy. Since ML algorithms are, in practice, deployed on finite computational resources, penalizing overly complex models is necessary.

$$obj(\vec{\theta}) = L(\vec{\theta}) + \Omega(\vec{\theta}) \tag{4.4}$$

The new objective function can then be written schematically as shown in equation 4.4 [33]. Here, $L(\vec{\theta})$ is a loss function, and $\Omega(\vec{\theta})$ is the regularization term. The vectors $\vec{\theta}$ are the parameters of the model that must be optimized. While the loss function concerns itself with individual data points, the regularization term concerns itself with each decision tree in the ensemble. So, the loss function will be summed over all data points, whereas the regularization will be summed over each tree in the ensemble. If we take the functions within the CART space, $f_k(\vec{x_i})$ as in equation 4.3, we can then express the overall objective function more concretely as equation 4.5 [32].

$$obj(\vec{\theta}) = \sum_{i=1}^{n} L(y_i, \hat{y_i}^{(t)}) + \sum_{k=1}^{t} \omega(f_k) \tag{4.5}$$

Where n is the total number of data points and t is the total number of individual trees in the ensemble. In this new formulation, $\omega(f_k)$ is referred to as the complexity score of the model – simpler decision trees will have a lower complexity score. While a more complex model will likely lower the loss function, it will also raise the complexity score of the trees and therefore may not necessarily minimize the objective function. This objective function, then, must be minimized; the inclusion of both the loss and regularization term allows the optimal balance between accuracy and complexity to be reached.

In equation 4.5, one can notice that the prediction has been given a superscript (t). This prediction, $\hat{y_i}^{(t)}$ denotes the predicted value at the tth step in the training process. In XGBoost,

each step t introduces one additional decision tree to the ensemble. This addative process leads to two different avenues to control the complexity of the model; each individual decision tree may have fewer nodes, or the total number of trees may be reduced. The prediction score at each step can then by built up iteration by iteration. A starting point is chosen, then each additional tree added to the ensemble seeks to further refine the model. Of course, the parameters of each tree, $\vec{\theta}$, can be optimized at each step to minimize the objective function. This additive approach is shown iteratively in equation 4.6, where the connection to the previous definition in equation 4.3 is obvious.

$$\begin{aligned} \hat{y}_i^{(0)} &= 0 \\ \hat{y}_i^{(1)} &= f_1(\vec{x}_i) = \hat{y}_i^{(0)} + f_1(\vec{x}_i) \\ \hat{y}_i^{(2)} &= f_1(\vec{x}_i) + f_2(\vec{x}_i) = \hat{y}_i^{(1)} + f_2(\vec{x}_i) \\ &\cdots \\ \hat{y}_i^{(t)} &= \sum_k^t f_k(\vec{x}_i) = \hat{y}_i^{(t-1)} + f_t(\vec{x}_i) \end{aligned} \tag{4.6}$$

Combining this with the defintion of the objective function in equation 4.5 gets to the heart of the XGBoost boosted decision tree algorithm; at each iterative step, a new tree must be added that seeks to further minimize the objective function. To best elucidate the iterative construction of an optimized tree, the objective function at each step must be quantified. The definition of the tth prediction given in equation 4.6 can then be added to the definition of the objective function given in equation 4.5. This yields the expression in equation 4.7.

$$obj(\vec{\theta})^{(t)} = \sum_{i=1}^{n} L(y_i, \hat{y}_i^{(t-1)} + f_t(\vec{x}_i)) + \sum_{k=1}^{t} \omega(f_k) \tag{4.7}$$

Now, of course, the loss and regularization terms must be defined and understood such that the objective at each step can be concretely defined in terms of the model parameters, $\vec{\theta}$.

For the case of a logistic loss, an analytical form of the iterative loss function is rather complex, especially if a summation over it is desired. Instead, the taylor expansion of the loss function can be employed. It is generally a sound procedure to only keep up to the quadratic term in the taylor expansion [33]. The expansion up to the quadratic term can be seen in equation 4.8. As they will be used in subsequent steps, the first and second derivative terms are defined as g_i and h_i, respectively (as seen in equation 4.9).

$$obj(\vec{\theta})^{(t)} = \sum_{i=1}^{n} [L(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(\vec{x}_i) + \frac{1}{2} h_i f_t^2(\vec{x}_i)] + \sum_{k=1}^{t} \omega(f_k) \tag{4.8}$$

$$\begin{aligned} g_i &= \partial_{\hat{y}_i^{(t-1)}} L(y_i, \hat{y}_i^{(t-1)}) \\ h_i &= \partial_{\hat{y}_i^{(t-1)}} L(y_i, \hat{y}_i^{(t-1)}) \end{aligned} \tag{4.9}$$

This general form allows for any arbitrary loss function to be optimized; as long as the change in the loss function can be quantified, the constants g_i and h_i can be determined and the objective can be updated at that step. This powerful formulation allows XGBoost to calculate an updated objective relatively rapidly for each new tree added to the ensemble, which therefore allows the training process to be closely monitored.

Before the final objective function can be quantified, the regularization term must be understood. The complexity functions, $\omega_k(f)$, must be expanded, then. In order to do this, an understanding of the form of the individual tree functions, $f_t(\vec{x})$, must be achieved. As shown in the simplified schematic in figure 4.2, these functions are a collection of weights assigned by each node in the tree. While the assignment of these weights is relatively complex, the tree itself is a simple addition of the weights. Given some function that assigns the weights at each node of the tree, $q(\vec{x})$, the tree function can be defined as in equation 4.10. This $q(\vec{x})$ is usually called the tree structure function, as it determines the splitting of the tree into different leaves and how the weights are assigned at each leaf.

$$f_t(\vec{x}) = w_{q(\vec{x})} \tag{4.10}$$

If we take T to be the total number of leaves in the individual tree, then the weights will be T real numbers. Larger weights lead to larger changes in the total output score, so the tree should generally try to reduce the swings in output by restricting these weights to be lower. It then becomes clear that there are two sources for complexity within a tree ensemble that must be controlled – the total number of leaves increases the complexity, as does the total sum of weights. This suggests the form of the complexity function given in equation 4.11.

$$\omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \tag{4.11}$$

These two new values, γ and λ, are known as the L1 and L2 regularization terms. These tunable parameters are usually considered hyperparameters to distinguish them from the other parameters in the model; while parameters are adjusted at each step to minimize the objective function, hyperparameters are fixed before the training begins as a constant throughout the training processes and apply to all trees equivalently. There are additional hyperparameters to an XGBoost BDT model, which will be discussed further in chapter 5.

With the updated form of $f_t(\vec{x})$ and $\omega(f)$, the full objective function at a given step can be written as equation 4.12 [33].

$$obj(\vec{\theta})^{(t)} \approx \sum_{i=1}^{n} [g_i w_{q(\vec{x})} + \frac{1}{2} h_i w_{q(\vec{x})}^2] + \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \tag{4.12}$$

Immediately, it can be seen that there is a sum over the index j runs over all of the leaves in the tree from j = 1 to T, whereas the sum over n sums all of the data points at each leaf, j. This can then cleverly be written into an easier to digest form, where terms with the same powers of ω_j can be grouped. The set $I_j = \{i|q(\vec{x}) = j\}$ can be defined as the set of indices where the data point is in the specific leaf, j. This makes sense, as each leaf in the tree will not use all input data

points. Returning to the example in figure 4.2, it is clear that three of the data points are split at the first leaf, leaving only two that make it to the second leaf. So, then, the data points that are not involved in the second leaf need not be summed over for that leaf when determining the objective function. This means that the final objective function can be rewritten as a sum over all leaves, j, as given in equation 4.13.

$$obj(\vec{\theta})^{(t)} \approx \sum_{j=1}^{T} [(\sum_{i \in I_j} g_i) w_j + \frac{1}{2}(\sum_{i \in I_j} h_i + \lambda) w_j^2] + \gamma T \tag{4.13}$$

Finally, these sums over the index i can be defined as their own value for simplicity. Since for each j there will be one value of this sum over i, the quantities $G_j = \sum_{i \in I_j} g_i$ and $H_j = \sum_{i \in I_j} h_i$ can be defined. This simplification of notation allows the final form of the objective function to be written as in equation 4.14.

$$obj(\vec{\theta})^{(t)} \approx \sum_{j=1}^{T} [G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2] + \gamma T \tag{4.14}$$

Differentiating this with respect to w_j and finding where it is 0 allows the individual leaf weight that minimizes the objective function to be found as given in equation 4.15. Then, these weight can be put into the objective function to find the maximum reduction in objection function for a single tree iteration, t. This is shown in equation 4.16.

$$\omega_j^* = -\frac{G_j}{H_j + \lambda} \tag{4.15}$$

$$obj^* = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T \tag{4.16}$$

The form of this equation makes the logic of the minimization process clear. The first term depends on H_j and G_j, which are determined by the rate of change of the loss function. So, an update that more rapidly changes the loss function will more rapidly reduce the objective function given that this term is negative. The second term simply shows that making a more

complex tree with more leaves will increase the objective function. A balance, therefore, will be specifically found by the algorithm that helps maximally reduce the loss at each step while still ensuring that the complexity of the model is kept relatively low.

4.2.2 Tree Boosting Algorithm

So, with equations 4.15 and 4.16, there exists a way to evaluate how well-optimized a specific tree structure, $q(\vec{x})$, is. How, though, are the possible structures explored? While the above method would work well if all structures could be evaluated and the optimum structure selected, this is not at all practical in a computational sense. Given that BDTs can be trained on hundreds of thousands to millions of events with dozen of input variables and possibly thousands of trees in the ensemble, an efficient approach to additive tree boosting must be found.

The most straightforward algorithm is called the exact greedy algorithm [33]. This algorithm begins with one single leaf, then iterates over all possible splits at that leaf. This process is repeated until either a maximum depth is reached, or additional splits no longer improve the objective function. The issue, therefore, changes from trying to directly find the optimum tree structure to simply evaluating the possible splits at each node, choosing the optimum split, then iterating. While equation 4.16 is useful to evaluate an entire tree, it does not directly help with the splitting process. Instead, a new form of the objective function must be determined in terms of the proposed split.

Starting from the general objective score found in equation 4.16, a split can be defined. Since the chosen tree structure is a binary decision tree, there are only ever two possible branches created at each node. These branches can be termed left and right, for convenience. Before, the set $I_j = \{i|q(\vec{x}) = j\}$ was defined such that all data points y_i within a particular node were contained within it. A similar set can then be defined for the left and right side of side of the split, denoted by I_L and I_R, respectively. Then, the terms G_j and H_j can be redefined in terms of these sets for a single value of j as given in equation 4.17.

$$G_L = \sum_{i \in I_L} g_i \qquad G_R = \sum_{i \in I_R} g_i$$
$$H_L = \sum_{i \in I_L} h_i \qquad H_R = \sum_{i \in I_R} h_i \tag{4.17}$$

This allows the split objective function to be written as in equation 4.18 [33]. Of note is the fact that this is the objective contribution from a single split, so T = 1 in this case. Additionally, the terms G_I and H_I are taken to be the sums over the entire set I before the split.

$$obj_{split} = \frac{1}{2}[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_I^2}{H_I + \lambda}] + \gamma \tag{4.18}$$

In practice, this form of the objective function is actually employed by the XGBoost algorithm. Using this minimization process, individual leaves can be added to each tree such that the gain in accuracy and the gain in complexity are adequately balanced for each additional split added.

There are two additional ways in which overfitting (i.e. overcomplexity) of the model is avoided. One applies to the individual weight values, whereas the other applies to each tree. Shrinkage was first described by Friedman, and is a factor (denoted by η in XGBoost) applied to every weight within every tree [31]. This shrinkage aims to slow down the learning process, allowing the contribution of each tree to be controlled and therefore allowing models with more individual trees that each have a smaller impact to avoid overfitting. The learning rate, η, is another hyperparameter of the model in XGBoost – it is tuned once for each model. The effects of the learning rate will be described in more detail in chapter 5.

The other way to avoid overfitting is to restrict the set of input features used by each tree. Taking a simple vector of d input features, $\vec{x_i} = \{x_1, x_2, x_3 ... x_d\}$, this vector can be sampled differently for each tree in an ensemble. So, where the first tree in the ensemble may take its input features as $\vec{x_i^{(1)}} = \{x_1, x_2, x_3\}$, the second tree may take $\vec{x_i^{(2)}} = \{x_{d-2}, x_{d-1}, x_d\}$. This further

MVA - ACT

limits the power of each individual tree and the redundancy of each tree in the ensemble, thereby lowering the chance of overfitting. The proportion of all available input features that should be taken at each tree is an additional hyperparameter that can be set for the whole ensemble.

Chapter 5

Development of an MVA-Based High Level Trigger

5.1 Limitations and Benefits of Current $H \rightarrow \gamma\gamma$ Trigger

As discussed in section 3.2, the standard $H \rightarrow \gamma\gamma$ HLT (sHLT) uses hard cuts on kinematic, isolation, and shower-shape variables to select events. While cuts on some of these variables – such as R_9 and H/E – serve to remove lower quality photon candidates, others reduce the phase space of the trigger while not necessarily increasing the quality of the selected events. Principally, the current HLT makes rather tight cuts on the p_T of the two photon candidates. These cuts significantly reduce the trigger rate, as many background processes produce showers with low p_T. Many signal events, however, have at least one candidate with low p_T. It will be shown in section 5.3 that these p_T cuts significantly decrease the acceptance of simulated Monte Carlo signal events. Since many future analyses will be searching for either very rare processes or physics beyond our current understanding of the standard model, constraining the phase space of available information may affect the collaboration's ability to probe novel processes. Additionally, maximum kinematic coverage of the entire phase space allows the kineamtic distributions of the Higgs to be more completely measured. For example, searches for high-p_T Higgs and better measurements of associated production modes may benefit from more complete kinematic coverage.

The relative simplicity of the standard HLT, however, provides multiple benefits. Firstly,

the numerical cuts can be closely examined and understood very easily. Though the cuts have been relatively stable in time, they may adjusted and reexamined with relative ease. The effects of cuts on individual variables can be quantified, which allows maximum transparency into how a decision is made on an event-by-event basis. Secondly, a decision for each event can be made very rapidly. As discussed in section 3.2.2, minimizing computation time for each event is vital given the huge data rate and number of triggers deployed at the trigger farm. Since all of the quantities used by the current HLT are already being saved for each event, there is very little additional computation needed to arrive at a triggering decision. Any proposed changes to the $H \to \gamma\gamma$ trigger must have minimal impact on decision speed given the stringent restrictions on computation time.

5.2 Development of a Boosted Decision Tree for Photon Candidate Classification

The HLT is a binary classification problem, where the two classes are signal events to keep and background events to discard. While the HLT is a per-event selection, $H \to \gamma\gamma$ events contain two photon candidates. As described in 3.2, the sHLT first makes requirements on individual photons, then applies a final filter that considers both photons together. Our MVA-HLT uses a similar philosphy; a binary classifier can be used to calculate an identification score for each photon, then combined cuts can be made on these scores for two photon candidates. The process of creating the MVA-HLT, then, is twofold. Firstly, the single photon binary classifier must be constructed such that signal-like photon candidates are given a high score and background-like photon candidates are given a low score. Secondly, cuts on both photon candidate scores must be selected such that the signal efficiency is maximized while the data rate is minimized.

5.2.1 XGBoost-Based BDT Software

XGBoost is a commonly-used software package throughout CMS, as it excells at binary classification problems. Specifically, Boosted Decision Trees (BDTs) are used commonly in

CMS software. BDTs provide a good balance between model complexity and transparency. By tuning a relatively small number of model hyperparameters, as discussed in section 5.2.4, model complexity can be controlled and understood. For the development of this BDT, the python version of XGBoost was used, though the final implementation of the model must be done in the C-based CMSSW (as discussed in section 5.5).

5.2.2 Signal and Background Samples for Classification

The first – and perhaps most important – step in creating a binary classifier is selecting samples on which to train the BDT. It is important that the signal sample chosen accurately represents the type of events that should be kept at the HLT, while the background sample must encompass all possible source of background events.

Signal Samples

Monte Carlo simulations exist for various physics processes that contribute to $H \to \gamma\gamma$. While there are several common production modes for the Higgs boson (as discussed in section 1.2), gluon fusion Higgs production (ggH) has the largest production cross section. In addition to the large cross section, ggH production does not produce as many signatures in addition to the decaying Higgs as other production modes. For these reasons, ggH signal MC provides a clean signal that maximizes the photon discrimination ability of a BDT.

In order to validate the performance of the model, the acceptance of these signal MC events must be quantified. As a result, half of the sample is used for training, while the other half is reserved for later validation. Events are split by event number into two separate categories: even event nubers are used for the training sample, and odd event numbers will be used for the validaiton of the model performance.

Background Samples

At the trigger level, background simulation poses a major challenge. Simulated background events for $H \to \gamma\gamma$ fall into three major categories: events containing two photon

candidates, events containing a photon and a jet, and events containing two jets. While it is possible to simulate these background sources, the simulation – especially of the jet-jet events – is challenging and therefore may not perfectly allign with the real data.There are also backgrounds which are not simulated, like background from (ASK!) beams. The most obvious resolution to this issue is to simply treat the data sample as background. While, of course, the goal of the trigger is to preserve signal signatures from the data, it has been discussed in chapter 1 that the overall expected count of signal events in this data sample is relatively low. As long as a clean signal sample is provided to the BDT, the incredibly small percentage of possible $H \rightarrow \gamma\gamma$ events in the data should not appreciably harm the model's ability to differentiate signal and background photon candidates. Additionally, the goal of the trigger is not to select final signal events for analysis. Instead, the aim is to simply reduce the overall data rate by a large factor, meaning that using the data as background for the BDT is a sound strategy. As with the signal MC, even event numbers are used for the training, while odd event numbers are reserved for later validation of model performance.

5.2.3 Determination of Input Variables

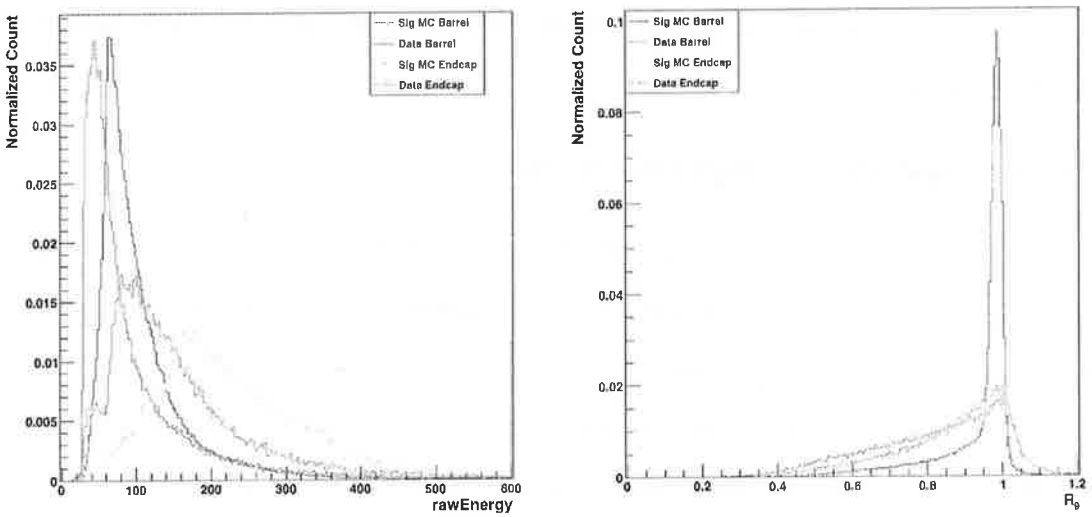
After signal and background samples are chosen, input variables for the BDT must be chosen to optimize the separation of signal from background. The most obvious starting place for this is by analogy to the sHLT; since cuts on the kinematic, isolation, and shower-shape variables have been successfully used in the preexisting trigger, a BDT should be able to use these variables to even greater effect. In table 5.1, the chosen input variables are listed along with their definitions.

The chosen variables are largely shower-shape and isolation variables, as these can be used to differentiate a legitimate photon signature from a jet shower. A major goal of the BDT is to increase kinematic coverage; relying primarily on shower-shape and isolation variables ensures that the selection is not biased towards a specific kinematic region. Next, plots of the BDT input variables can be examined, where the signal MC is shown in dark (barrel) and light

Table 5.1. Table of BDT input variable names, and a short description of each.

Variable Name	Description
Raw Energy	The total energy deposited in the 5x5 supercluster
R_9	The proportion of the energy contained in the central 3x3 crystals relative to the raw energy
$\sigma_{i\eta i\eta}$	The covariance in terms of crystal spacing (i_η)
η_{SC}	The eta coordinate of the center of the supercluster
η Width	The supercluster width in η
ϕ Width	The supercluster width in ϕ
H/E	The ratio of the total supercluster energy measured in the HCal to that measured in the ECal
ECalPFIso	The isolation energy of the particle deposited in the ECal

(endcap) blue, while the data is shown in red (barrel) and pink (endcap). These distributions are self-normalized, so the y-axis value corresponds to the ratio of events in that bin versus the entire distribution.

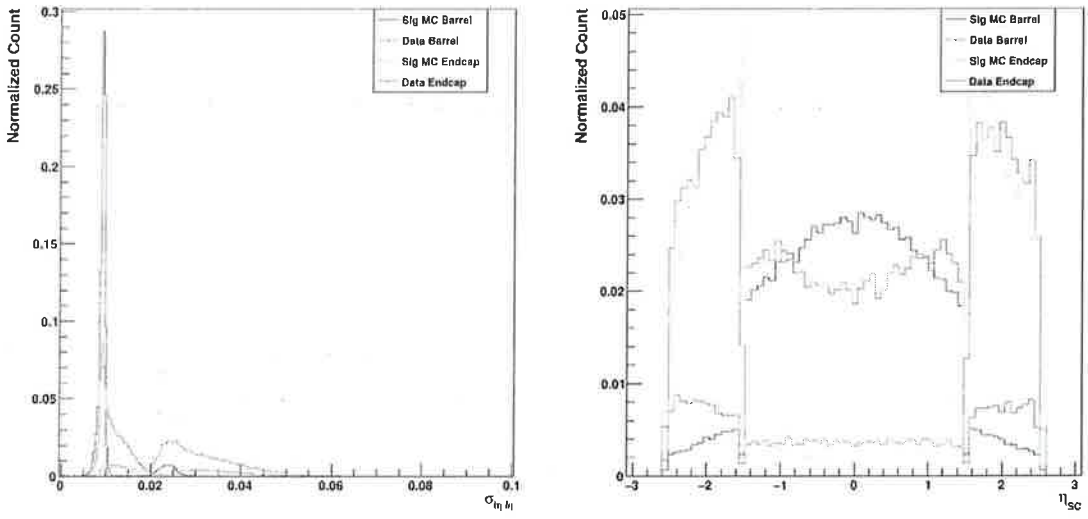


(a) Raw Energy **(b)** R_9

Figure 5.1. Plots of raw supercluster energy and R_9. The signal MC is shown in dark blue for the barrel and light blue for the endcap, whereas the data (background) is shown in red for the barrel and pink for the endcap.

The raw supercluster energy gives a measure of the total energy deposited in the ECal supercluster. In figure 5.1a, it can be seen that the signal MC has a slightly higher center than the

background in the barrel. This difference is more pronounced in the endcap, however, where the signal MC has a considerably higher average raw energy. The separation in these two detector regions is not major, however. The R_9, on the other hand, proveds to be a very well-separated training variable. For both the barrel and endcap, the signal distributions seen in 5.1b are very well-peaked near 1. This means that most of the signal MC events have a very large percent of their energy deposited in the central 3x3 crystals of the supercluster. This is often the case for real photons, as their geometric extent is smaller than that of jets. The data, however, is a much broader distribution. While it peaks near 1, there are still many photon candidates with R_9 approaching lower values. Since QCD jets tend to have a larger geometric extent, this is expected. As a result, this is a very strong discrimination variable for the binary classifier BDT.



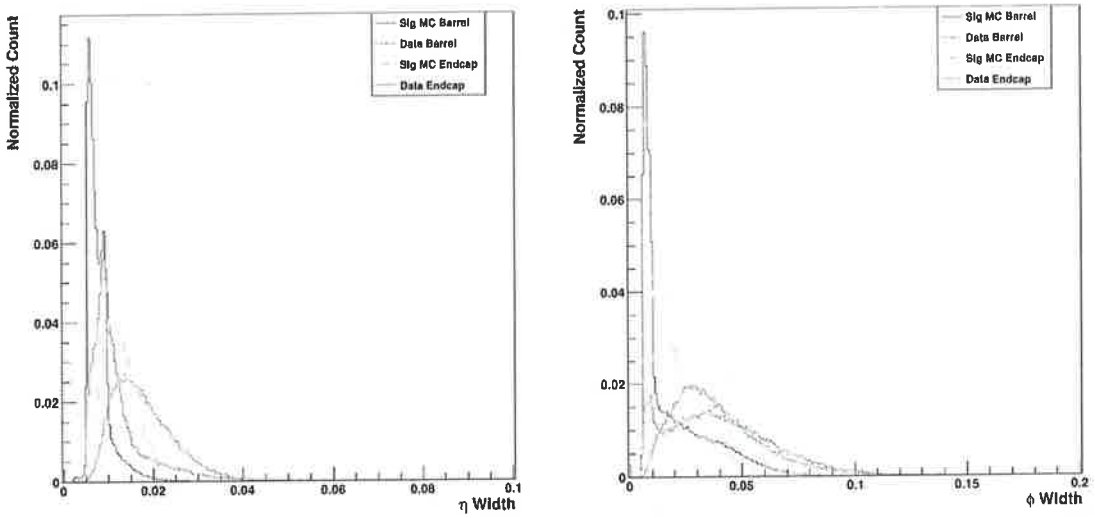
(a) $\sigma_{i\eta i\eta}$ **(b)** η_{SC}

Figure 5.2. Plots of the $\sigma_{i\eta i\eta}$ and η_{SC}. The signal MC is shown in dark blue for the barrel and light blue for the endcap, whereas the data (background) is shown in red for the barrel and pink for the endcap.

The $\sigma_{i\eta i\eta}$ is another shower-shape variable that has powerful discrimination abilities. As can be seen in figure 5.2a, the barrel and endcap are rather well-separated for both the MC and data. This is due to the difference in crystal positioning in the two detector regions; since the

endcap crystals are larger with larger spacing as described in section 2.2.2, the σ in terms of the crystal spacing tends to be larger. Additionally, it is immediately obvious that the photon candidates from the signal MC are much more sharply peaked in both the barrel and endcap relative to the data events. Given that many of the data events contain QCD jets, this is to be expected.

The η_{SC} can help provide information on the kinematics of the event, as different reactions and decays will be arranged differently. While the barrel and endcap are self-normalized here, the relative difference in shapes for signal and background in both the barrel and endcap can be examined. In the barrel, it is clear that the single Higgs signal MC tends to be more central, while the data sample eschews this region in favor of the higher-η regions of the barrel. These kinematic differences are less pronounced in the endcap, where the signal MC and data shapes are relatively similar.

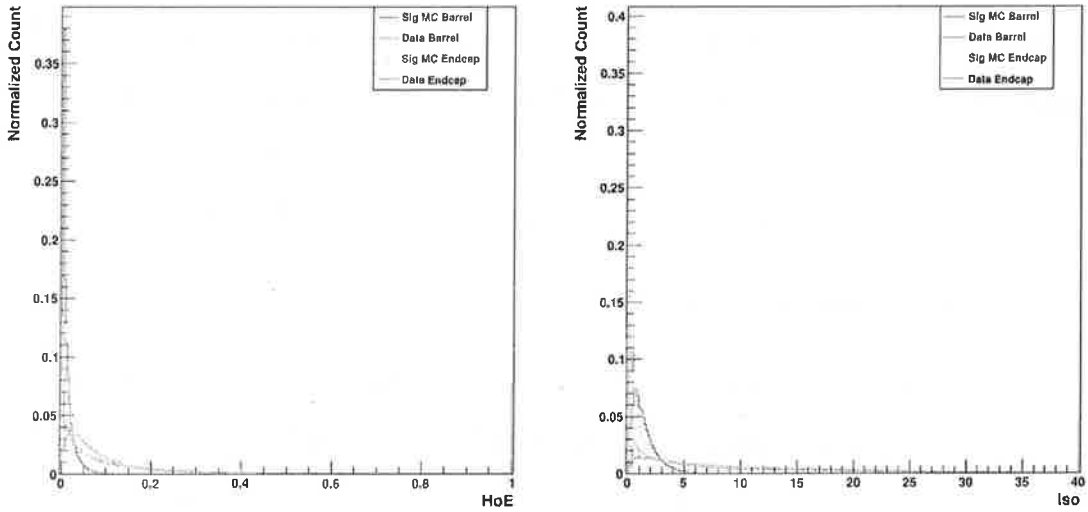


(a) ϕ Width **(b)** η Width

Figure 5.3. Plots of the η and ϕ widths. The signal MC is shown in dark blue for the barrel and light blue for the endcap, whereas the data (background) is shown in red for the barrel and pink for the endcap.

Next, the η and ϕ widths (in figure 5.3)are shower-shape variables that provide further

information of the geometric extent of the ECal energy deposition. As with the $\sigma_{i\eta i\eta}$, the expected shapes of these distributions are affected by the crystal geometry – as a result, the endcap and barrel distributions are expected to differ. Still, the signal MC and data can be seen to show some separation within the barrel and endcap. On average, the data has higher width in both η and ϕ relative to the signal MC.



(a) H/E (b) Iso

Figure 5.4. Plots of the H/E and isolation. The signal MC is shown in dark blue for the barrel and light blue for the endcap, whereas the data (background) is shown in red for the barrel and pink for the endcap.

The H/E in figure 5.4a is a powerful variable for separating photon-like ECal signatures from those originating from QCD jets. Measuring the ratio of how much energy is deposited in the HCal versus the ECal gives some idea of how likely to be a hadronically-produced signature the photon candidate is. Theoretically, a true photon would deposit purely in the ECal, leaving virtually 0 energy signature for the HCal behind it. As a result, real photons should have and H/E near 0. Clearly, the figure shows that this is largely true for the signal MC, whereas the data events – which are largely composed of jet-like candidates – have on average much higher H/E values. In fact, there are very few data events with an H/E in the first bin, while nearly 40% of

signal events exhibit this behavior. As a result, this is a very strong separation value for the BDT.

The isolation, measured in units of energy (GeV), helps measure how much additional energy is deposited outside of the central supercluster. For jet-like events, this value is expected to be higher than true photon signatures; since QCD processes generally produce mutltiple wide jets together, it is expected that much energy will be deposited immediately near the supercluser. Real photons, on the other hand, are more likely to provide a clean energy deposition with little additional surrounding sources of energy. As seen in figure 5.4b, this truth is borne out by the difference between the signal MC and the data. While there are still data events in the first bin near 0 isolation energy, there are nearly 3x as many events in this bin in the signal for both the barrel and endcap. This illustrates the power that this variable provides the BDT.

5.2.4 Hyperparameter Tuning

Once the input samples and variables are determined, the technical details of the model must by optimized. The structure of the BDTs is determined by the hyperparameters of the model as discussed in chapter 4. The most fundamental hyperparameters are the maximum depth, the learning rate, and the total number of trees in the model. The max depth and number of trees directly determine the complexity of the model. A model with more trees will, based on the boosted gradient descent method discussed in chapter 4, more precisely estimate the function that maps the chosen input variables from the training sample onto the correct labels of signal versus background. To mitigate the risks of overtraining the model, XGBoost early stopping is used to determine the number of trees. As the software adds additional trees to the ensemble, it measures the chosen objective function at each step on a testing sample not used in the training. Early stopping ends the training of the model when the chosen metric hasn't improved in a chosen number of rounds. For this training, 50 early stopping rounds were used and the area under the curve (AUC) was used as the target metric. So, then, this hyperparameter is essentially fixed while the others must be optimized.

The maximum depth controls the complexity of each individual tree in the ensemble. As

discussed in chapter 4, the depth determines the number of layers each tree is able to employ. Deeper trees generally allow more precise mapping from input values to output scores at the cost of computation time; each layer of depth constitutes another set of weights that must be computed for each tree. Since it is vital that the computation time is minimized at the trigger level, this is a major consideration. Deeper trees also tend to overfit the training data more, as there are essentially more degrees of freedom with which to fit the mapping function. So, the max depth must be kept as low as possible to avoid both overfitting and unreasonable computation time.

The learning rate controls the size of the individual feature weights from each new tree added to the ensemble. A higher learning rate allows each tree to have higher weights, which will therefore affect the model performance more drastically. Lowering the learning rate makes the model more conservative, as it reduces the overall change to the model predictions provided by each additional tree. Having a higher learning rate generally reduces the number of trees necessary to converge, but decreases the accuracy since statistical fluctuations within each tree can be assigned higher weights.

It was found that the max depth and learning rate are best optimized in tandem; as the trees become more complex (higher max depth), it was found that making the individual steps more conservative (lower learning rate) led to less overfitting and higher performance. As a result, a 2D grid search was used to optimize these two hyperparameters. A range of both max depth and learning rate was defined, then a model was trained for each combination. The resulting models were appraised in terms of not only their objective scores (AUC and logistic loss in this case, both described in chapter 4), but also in terms of the number of trees and the overall model size in megabytes. Since the model must be loaded into RAM at the trigger farm and then the weights applied to each photon in each incoming data event, a model taking up more space will increase both the computation time and memory load at the trigger level.

After many trials, the best balance of model performance, minimization of overtaining, and model size was found to be a max depth of 7 and a learning rate of 0.25. Experimentally,

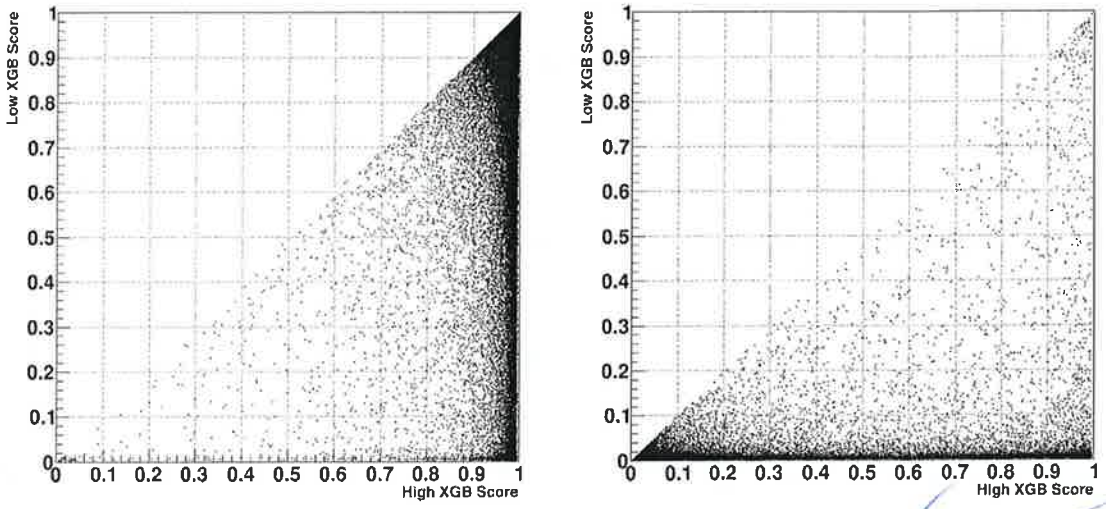
it was determined that many of the other hyperparameters had minimal impact on the model performance and size, so they were therefore left at their default values.

5.3 Determination of BDT Cuts

After the tuning and training of the model was optimized, a score is generated for each photon in each event. Each event may contain several photon candidates, but the job of the HLT is to determine if any pair of candidates exists that plausibly originates from $H \rightarrow \gamma\gamma$. Given that the BDT was trained such that photons originating from $H \rightarrow \gamma\gamma$ (MC) are assigned a high score and background processes are given a low score, the photon candidates are first sorted by the output BDT score. Then, the highest two scores can be selected and the values analyzed for both signal MC and data events.

Since the HLT path simply renders a decision about whether an event contains a $H \rightarrow \gamma\gamma$ process, cuts on the pairs of output scores must be chosen such that the number of signal events kept is maximized (high signal efficiency) while the total number of data events kept is minimized (low trigger rate).

In figure 5.5, the high and low scores for signal MC are shown at left. Since these are purely signal events, it is expected that the scores to be clustered near 1. As can be seen, the higher of the two scores (along the x axis) tends to be above 0.9 for a large percentage of events. Meanwhile, while the lower of the two scores does cluster above 0.8, there is still a large band of score values extending down to 0. This means that there are many signal events for which one photon candidate has a very high score, but the second photon candidate has a lower score.



(a) (b)

Figure 5.5. XGBoost output scores for two selected photon candidates in the Winter 2024 ggH signal MC (a) and 2023 Data (b). The higher of the two scores is plotted on the X-axis, while the lower of the two is plotted on the Y-axis.

Next, consider the scores of the data sample shown at right in in 5.5. Given the rarity of $H \to \gamma\gamma$ events at the trigger level, it is expected that essentially all of these data events correspond to background processes. As a result, the vast majority of these data photon candidates should be assigned a BDT score close to zero. As can be seen, the lower of the two photon scores very rarely extends above 0.1. Though the higher of the two scores is often close to 0, there are still many events where the value extends towards 1. Since this BDT is designed to indentify individual photon candidates, it can be inferred that many of these data events have one real photon (which is a major source of background as discussed in section 5.2.2).

As discussed in section 5.2.1, the single photon BDTs are trained separately for photon candidates in the barrel and those in the endcap. Due to the separate trainings – as well as the differences in kinematics and detector geometry – the score distributions for barrel and endcap photon candidates are not perfectly equivalent. As a result, the chosen cuts are tuned separately for the barrel and endcap photons. This presents an issue, as diphoton events may contain two barrel photon candidates (BB), two endcap photon candidates (EE), or one photon candidate

in each region of the detector (BE). The simplest way to deal with this is to choose the single photon cuts by reference to the BB and EE distributions, then apply the chosen barrel and endcap cuts to all photons in either region.

Given the shapes of the output distributions in figure 5.5, it is clear that a single cut value for both photons will not achieve maximum signal efficiency. While many of the high score photons are clustered above 0.9, there are still many events with a second photon candidate that has a lower score. In order to preserve these signal events while still removing much of the background, cuts can be selected such that the cut value on the lower score depends on the higher score. For first photons with a very high score, a very loose cut can be applied to the second photon without increasing the trigger rate much. As the score of the first photon decreases, the cuts on the second photon can be tightened. In the case where the first photon has a relatively low score, a tighter cut is applied on the second photon to help reduce the number of events with two bad photon candidates in the data while still preserving much of the signal. Since the data at the trigger level is strongly dominated by di-jet or single photon events, these cuts can drastically reduce the data rate while still preserving much of the signal. These cuts were experimentally determined, and are tabulated in table 5.2.

Table 5.2. Chosen score cuts for BB and EE events

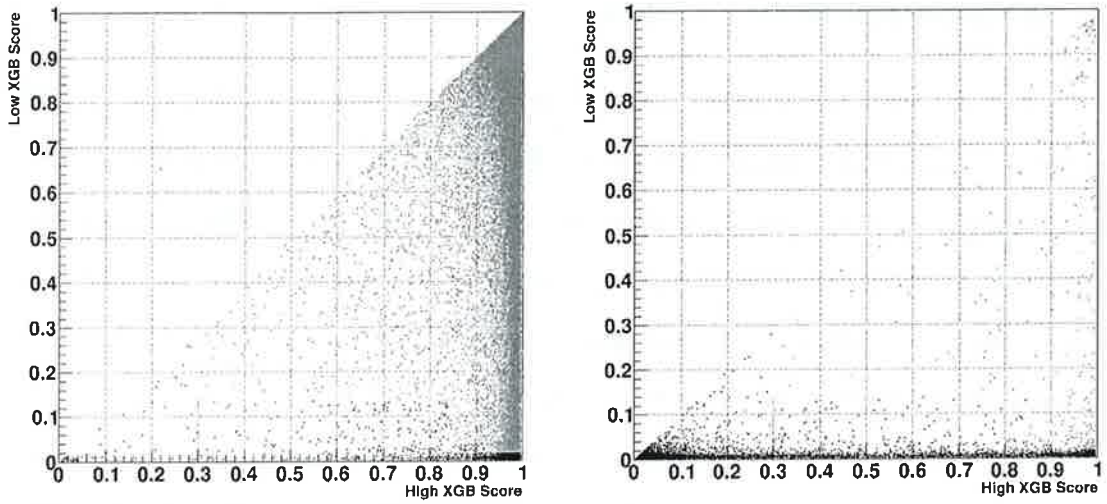
(a) Score cuts for BB events

High Score	Minimum Low Score
0.92	0.02
0.85	0.04
0.30	0.14

(b) Score cuts for EE events

High Score	Minimum Low Score
0.95	0.04
0.85	0.08
0.5	0.20

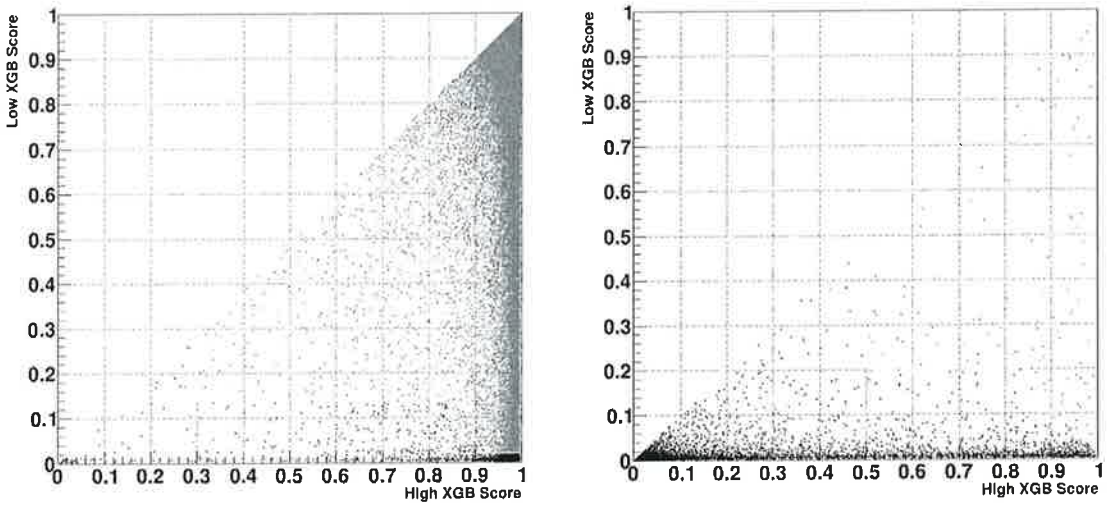
Events that satisfy these requirements can be plotted on top of figures 5.5 . In figures 5.6 and 5.7, the selected events for BB and EE can be seen, respectively, in red. Yellow lines are added to the plots to demarcate the cuts. These plots contain both the signal MC (left) and the Data (right).



(a) (b)

Figure 5.6. XGBoost output scores for two selected photon candidates in the Winter 2024 ggH signal MC (a), and 2023 Data (b). The higher of the two scores is plotted on the X-axis, while the lower of the two is plotted on the Y-axis.

BB XXX



(a) (b)

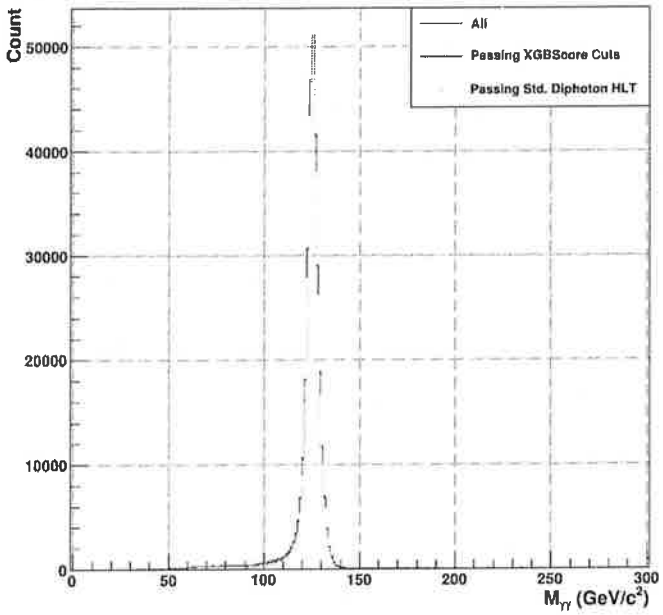
Figure 5.7. XGBoost output scores for two selected photon candidates in the Winter 2024 ggH signal MC (a), and 2023 Data (b). The higher of the two scores is plotted on the X-axis, while the lower of the two is plotted on the Y-axis.

EE Add XXX

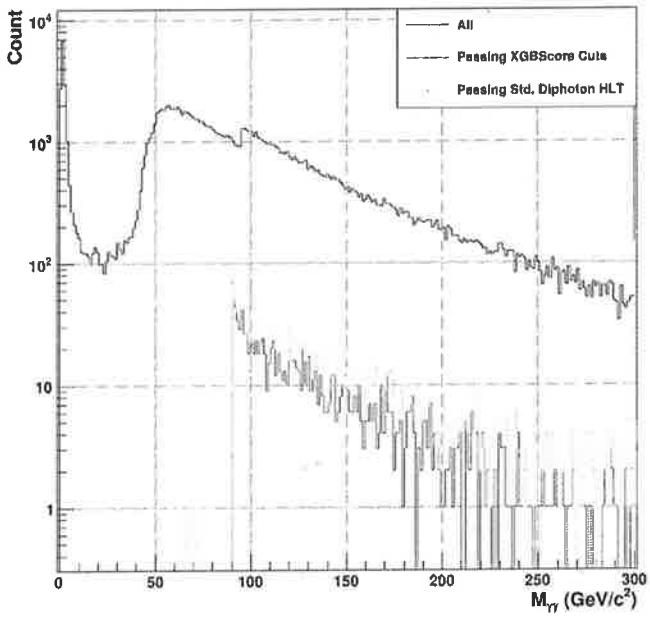
5.4 Evaluation of BDT Cuts

This choice of cuts, clearly, is able to preserve much of the signal MC while accepting very few of the data events examined. While the overall signal efficiency and data rate are key metrics to appraise our MVA-HLT, it is also important to examine where in the kinematic phase space events are being gained relative to the standard diphoton HLT. While a more complete measurement of this will be shown in chapter 7, a preliminary analysis can be made with the cuts applied in this initial development framework.

Next, plots of selected kinematic variables are shown. The events accepted by the current diphoton HLT are shown in green, while the events accepted by the new MVA-HLT are shown in blue. As with the previous score plots, the Winter24 GGH signal MC will be shown on top, whereas the 2023 Data will be shown on bottom.



(a)

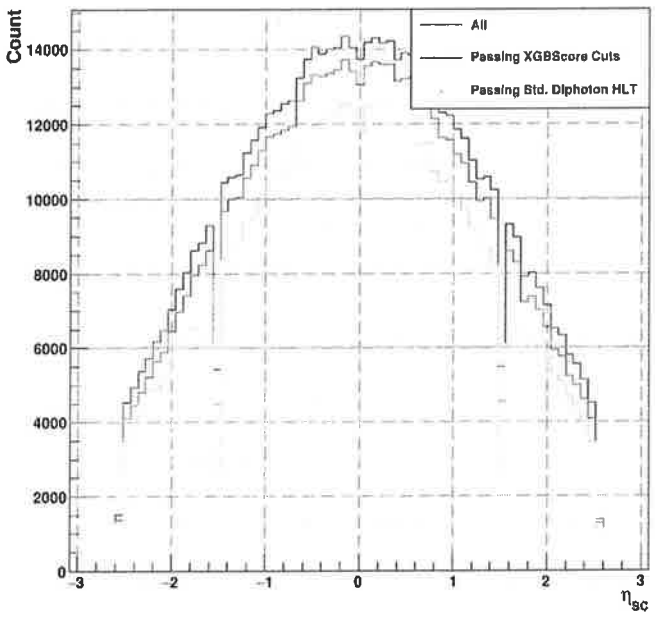


(b)

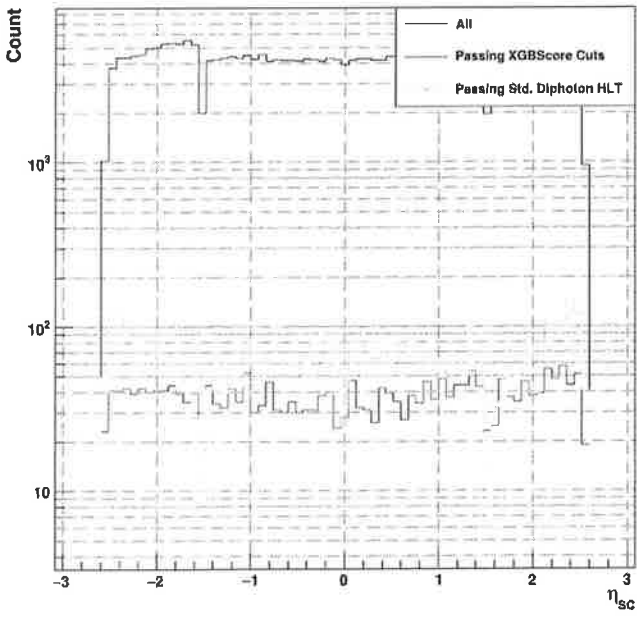
Figure 5.8. Diphoton mass for the signal MC (a) and data (b). Events accepted by the MVA-HLT are shown in blue, while events accepted by the sHLT are shown in green.

Beginning with the mass, it can immediately be seen that MVA-HLT accepts 94.2% of signal MC events, which is an approximately 13.6% relative gain in signal efficiency compared to the 82.9% acceptance of the standard diphoton HLT. Additionally, the data rate appears considerably lower; while the standard HLT accepts 1.44% of the analyzed events, the MVA-HLT only accepts 0.86%, which corresponds to a 40.3% reduction in data rate.

One major consideration when examining the mass is the measured mass width of the accepted signal events. Generally, a more narrow peak is considered a cleaner signal. By treating the mass distribution as approximately gaussian, the width can be easily understood by measuring the standard deviation. Since the distribution is only approximately gaussian, it is safer to instead construct the quantity $2\sigma_{effective}$, which is simply the width (in GeV/c^2) that contains 95% of the peak. Although this is not a statistically rigorous construct, it can be used to compare the approximate shapes of similar distributions in an interally consistent way. It can be seen that the $2\sigma_{effective}$ is marginally increased for the MVA-HLT, reaching 6.58 compared to the 6.07 of the events accepted by the standard trigger. Some increase in width is to be expected, as accepting more events and lower-p_T photon candidates will necessarily bring the distribution closer to the "all" curve that has a wider width than either selection.



(a)

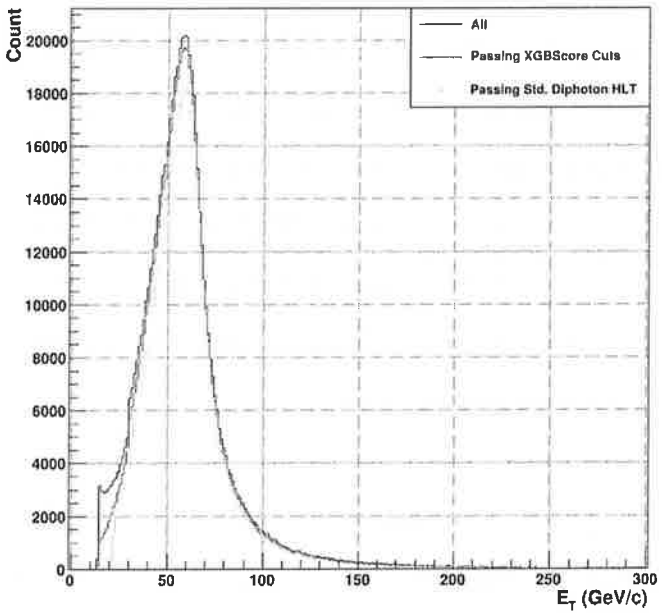


(b)

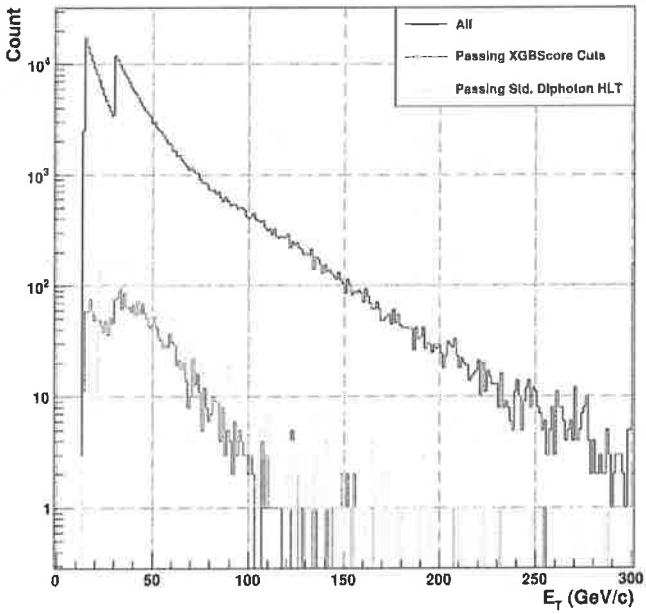
Figure 5.9. Photon η for the signal MC (left) and data (right). Events accepted by the MVA-HLT are shown in blue, while events accepted by the standard HLT are shown in green.

The next kinematic variable shown is the η_{sc} of both photons plotted together (figure 5.9). As seen in both signal and background, the separation between the ECal Barrel (EB) and ECal Endcap (EE) can be seen at $\eta_{sc} \approx \pm 1.5$. For the signal, thoughout the barrel, both the MVA-HLT and sHLT accept photon candidates in relatively constant propotion to the total distribution, though the MVA-HLT has overall improved acceptance. While the MVA-HLT still has relatively constant acceptance in the EE, it can immediately be seen that the sHLT is excluding many events in the lowest-η_{sc} region of the endcap. Investigations have shown that this is due to the R_9 cuts made within the trigger. This represents a reduction in kinematic coverage for the standard HLT, whereas the MVA-HLT is able to resolve this issue and maintain maximum kinematic coverage.

Considering the background, it can immediately be seen that the MVA-HLT has a relatively flat acceptance in both the EB and EE, while the standard HLT appears to accept more data at the extreme-η_{sc} reigions of the endcap. As can be seen from the MC, this region tends to contain less of the $H \rightarrow \gamma\gamma$ signal events and therefore represents a lower signal to background ratio.



(a)



(b)

Figure 5.10. Photon E_T for the signal MC (left) and data (right). Events accepted by the MVA-HLT are shown in blue, while events accepted by the standard HLT are shown in green.

Finally, consider the transverse energy of the photon candidates, E_T shown in figure 5.10. Here, both candidates are again shown on the same plot, which explains the double-peaking behavior in the background distribution. The sHLT contains cuts at 30 and 22 GeV for the lead and sublead photon, respectively. The sublead cut can clearly be seen in both the signal, at left, and the background at right. While the new MVA-HLT selection accepts more of these low-E_T photons, it is clear that little of the gain comes from this kinematic region. Below about 30 GeV, both selections tend to cut out many photons from the total distribution. In the case of the MVA-HLT, this is likely due to those photon candidates being of lower quality and therefore being assigned a lower BDT score.

In Table 5.3, the acceptance percentages are given for the 3 different diphoton η_{sc} configurations, as well as the total acceptance for both signal and background, compared to the sHLT cuts.

Table 5.3. Signal efficiencies and background acceptances for both triggers, broken down by diphoton η_{sc}

(a) Signal Efficiencies

Region	MVA-HLT(%)	Std. HLT(%)
B-B	95.93	89.32
B-E	91.91	74.10
E-E	89.46	76.85
All	93.86	82.64

(b) Data Acceptances

Region	MVA-HLT(%)	Std. HLT(%)
B-B	0.65	1.06
B-E	1.20	2.17
E-E	0.58	1.07
All	0.88	1.56

5.5 Final Implementation in HLT Menu

As discussed in section 3.2.3, triggering and event reconstruction is handled by the distributed CMSSW software framework. As a result, any proposed trigger developments must be deployed in a central CMSSW release. While the software strategy is the same as the original python implementation, the code must of couse be rewritten in C and adapted to work within the trigger.

As described in section 3.2, the HLT operates on all photon candidates in an event, then

simply renders an event-by-event decision if any pair of photons satisfy the chosen conditions. As a result, the first step of the implementation is to create a module that calculates the BDT score for each photon candidate in every event. Once these scores are stored within the trigger path, another module must be constructed to check if any possible pair of photon candidates passes the two-fold cuts described above. Importantly, due to the constraints on storage space, the scores of the individual photon candidates cannot be saved to the event. As a result, the only output from the MVA-HLT is a decision on whether the event should be accepted or rejected.

Luckily, these two modules are the only software additions required to add the BDT to the trigger menu. Beyond that, standard modules can be used (with some modifications to the cuts made) within ConfDB to construct the entire trigger path as discussed in section 3.2.3. The first step of the HLT path is to require the necessary L1 triggers. The standard L1 suite required by the sHLT is used here, which consists of a combination of single and double EGamma triggers as described in section 3.1.

In order to reduce the amount of MVA scores to be calculated, the very loose kinematic cuts are applied to all photon candidates in the event as the first step. While these cuts are quite loose, they still serve to reduce the number of candidates considered. Since many data events may have many photon candidates, this will help reduce the time taken to calculate the BDT scores.

With the preliminary cuts in place, the module that calculates the scores is placed in the HLT path, followed by the module that makes the cuts as described in section . Finally, a combined module is created that combines these BDT cuts on both photons with a mass cut of $M_{\gamma\gamma} > 90 GeV/c^2$. So, then, the whole HLT path serves to check if there is any possible pair of photon candidates that passes the loose kinematic cuts, the pairwise BDT cuts, and the $M_{\gamma\gamma}$ cut. If any pair satisfies these conditions, a flag is added to the event and all photon candidates are kept at the RAW level.

Chapter 6

MVA-Based Offline Preselection

As discussed in chapter 3, the HLT is one of the first steps in selecting data and MC events for analysis. Once a decision has been rendered by the HLT, the event is read out and saved in either the appropriate data stream or in the corresponding MC sample. Once these files are read in by the analysis software – in this case the Higgs Diphoton NanoAOD Analysis (HiggsDNA) software for Run 3 – further selection cuts are made to create a final analysis sample on which measurements can be made.

6.1 Current Diphoton Preselection

The current diphoton selection is created by direct reference to the standard diphoton HLT; first, p_T, shower-shape, and isolation cuts are made on individual photons in an event, then the single photons are paired together and cuts are made on the resulting diphoton. In the case of the HLT, a simple diphoton mass cut is made. At the offline analysis level, however, the diphoton cuts are made on $p_T/M_{\gamma\gamma}$.

The $H \to \gamma\gamma$ preselection is described in detail in [34]. Initially, loose cuts of $p_T > 20GeV/c$, $R_9 > 0.5$, $H/E < 0.08$, and $idMVA > 0.9$ are applied to each photon candidate. Additionally, the $|\eta_{sc}|$ is constrained to be below 2.5, and the gap region between the EB and EE is excluded ($1.444 < |\eta_{sc}| < 1.556$). Next, the electron veto is required to exclude photon candidates origininating from electron tracks. Beyond that, the remaining isolation and shower-